

# 智慧银行第四次实验报告

班 级：金融 202302

姓 名：夏雨欣

学 号：202301784

## 一. 实验目的

第一，理解软件工程全生命周期。而本实验通过 BMAD 方法论下的 PRD 撰写→架构设计→Epics/Stories 拆解→UX 设计→四个 Sprint 迭代开发→数据库适配→功能测试→Git 版本发布的完整链路，让参与者真正理解一个软件产品是如何从概念逐步落地的。

第二，掌握 AI 协作开发的核心范式。11 个实验均通过 Trae IDE 的 BMAD Agent 完成，从产品经理（PRD）到架构师（Architecture）到开发人员（Sprint 实施）到测试工程师（Playwright 测试），每个角色切换都由 AI Agent 完成。实验目的是训练“提出精确需求→AI 生成→人工审核迭代修正”这一人机协作闭环。

第三，建立金融科技产品思维。SmartBanking-CRM 系统涵盖客户管理、贷款审批、即申即贷、还款管理、数据看板等银行核心业务模块，通过亲手设计并实现这些功能（如 5 级客户等级体系、30 秒快速放款 SLA、3 级数据库降级方案），将课堂上学习的商业银行运营知识与实际系统结合，培养金融科技产品经理视角。

## 二. 实验环境

| 项目         | 配置                                                                           |
|------------|------------------------------------------------------------------------------|
| 操作系统       | Windows 11 (25H2)                                                            |
| IDE        | Trae (AI 智能编程助手)                                                             |
| Python 版本  | Python 3.x                                                                   |
| Node.js 版本 | Node.js v24                                                                  |
| 数据库        | SQLite (文件持久化)                                                               |
| 浏览器        | Microsoft Edge                                                               |
| 关键依赖       | pandas, numpy, matplotlib, statsmodels, tensorflow/keras, playwright, python |
| 版本控制       | Git + CNB (cnb.cool)                                                         |
| 测试后端       | http://localhost:3001                                                        |
| 测试前端       | http://localhost:5173                                                        |

## 三. 实验步骤

### 3.1 安装 BMAD

手动在终端中安装 BMAD

- Q 搜索 bmad-prd >
- Q 搜索文件 根目录 \_bmad >
- 深度思考
- Q 搜索 prd >

检查用户目录下是否有 \_bmad 目录

Get-ChildItem -Path "\$env:USERPROFILE" -Directory -Filter "\_bmad" -ErrorAction SilentlyContinue | Select-Object FullName

#< CLIXML

<Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04"><Obj S="progress" RefId="0"><TNd="0"><T>System.Management.Automation.PSCustomObject</T><T>System.Object</T></TN><MS><I64 N="SourceId">1</I64><S="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj><Obj S="progress" RefId="1"><TNRef RefId="0" /><MS><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj></Objs>

- 深度思考
- \_bmad 已在 C:\Users\28376\\_bmad ! 说明你手动安装成功了。让我先验证安装清单，然后启动 PRD 流程:
- ◎ 读取 manifest.yaml L1-End
- ◎ 读取 bmad-crm系统开发实验手册.md L205-264

深度思考

✅ BMAD v6.9.0 已安装确认! core + bmm 模块就绪。

3.2 创建产品需求文档

在 Trae CN 中执行 `bmad-create-prd` 命令启动 BMAD 产品经理 Agent。输入项目背景，商业银行 CRM，面向对公/零售业务，覆盖客户管理、贷款申请、审批流程、数据看板

AI Agent 逐模块引导完善：目标用户（4 类角色）、功能需求（4 大模块 24 项功能点）、非功能需求（性能/安全/可用性）。最后定义客户等级体系（L1-L5：普通→战略伙伴）与业务流程（贷款申请、等级评定）

"=Record"><AV>止在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj><Obj S="progress" RefId="1"><TNRef RefId="0" /><MS><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj></Objs>

深度思考

✅ PRD 生成完成!

| 项目 | 内容                                 |
|----|------------------------------------|
| 文件 | C:\Users\28376\_bmad-output\prd.md |
| 大小 | 7,219 字节                           |
| 章节 | 8 大核心章节                            |

8 大章节概览

| # | 章节     | 关键内容                                    |
|---|--------|-----------------------------------------|
| 1 | 产品概述   | 系统定位、4类目标用户、核心价值主张                      |
| 2 | 功能需求   | 客户管理(6项) + 贷款申请(5项) + 审批流程(6项) + 报表(4项) |
| 3 | 非功能需求  | 性能(并发5000+)、安全(加密+审计)、可用性(99.9%)        |
| 4 | 用户角色   | 普通客户/客户经理/风控审批员/管理员，含权限概要               |
| 5 | 客户等级体系 | L1-L5 五级：普通→白银→黄金→钻石→战略伙伴               |

3.3 创建技术架构设计

首先基于 PRD v1.0 执行 `/bmad-create-architecture`。随后 AI 架构师 Agent 输出分层架构：客户端层 (React+Vite) → 网关层 (Nginx+TLS1.3) → 后端服务层 (Express 4) → 数据层 (PostgreSQL+Redis+RabbitMQ) 绘制 Mermaid 架构图 + ER 图 (6 张核心表)。最后定义 API 接口规范 (5 组共 18 个端点)、安全架构 (JWT+RBAC+AES-256)、部署架构 (3 节点集群)

CodeBuddy

+ ⓘ ⚙ ...

我是 **Winston (架构师)**，基于已生成的 PRD 设计技术架构:

生成完整技术架构设计文档

📄 architecture.md (写入) +405 -0

查看文件 ↕

深度思考

确认架构文档生成

📄 Get-Item "C:\Users\28376\bmad-output\architecture.md" | Select-Object Name, Length, LastWriteTime | Format-List

t

#< CLIXML

<Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04"><Obj S="progress" RefId="0"><TN RefId="0"><T>System.Management.Automation.PSCustomObject</T><T>System.Object</T></TN><MS><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj><Obj S="progress" RefId="1"><TNRef RefId="0" /><MS><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj></Objs>

深度思考

✅ 架构设计完成!

| 文件                           | 大小        | 内容   |
|------------------------------|-----------|------|
| _bmad-output\architecture.md | 12,653 字节 | 8 章节 |

文档结构

文档结构

| # | 章节     | 要点                                                                   |
|---|--------|----------------------------------------------------------------------|
| 1 | 架构总览   | Mermaid 全栈架构图 (客户端→网关→后端→服务→数据→外部系统)                                 |
| 2 | 技术栈决策  | React 18 + Vite + Express 4 + Prisma + PostgreSQL + Redis + RabbitMQ |
| 3 | 模块划分   | 前端8页面7模块 / 后端6路由4服务                                                  |
| 4 | 数据库设计  | Mermaid ER图 + 8张核心表                                                  |
| 5 | API 设计 | 认证/客户/贷款/即申即贷/审批 5组共22个接口                                            |
| 6 | 集成方案   | 央行征信 + 公安部核验 + 短信网关, 含超时/重试/降级                                       |
| 7 | 安全架构   | TLS 1.3 + JWT + RBAC + AES-256 + 审计日志 + 脱敏                           |
| 8 | 部署架构   | 私有云 DMZ/应用/数据/管理 四层部署图 + 规格表                                         |

3.4 创建 UX 设计

基于 PRD + 架构执行 `/bmad-ux`。定义视觉风格：深蓝主色调（#1B3A5C）、银行级专业感、WCAG 2.1 AA 无障碍。设计 5 个页面线框图：登录/注册页 → 客户仪表盘 → 贷款申请页 → 审批管理页 → 客户 360° 视图。制定交互规范：表单验证规则（5 种字段）、5 种加载状态处理、5 种错误类型、4 种 Toast 通知。绘制用户旅程图（3 条）：新客户首次申请、L4/L5 即申即贷、客户经理日常跟进。情感映射：登记关键情绪时刻（等待焦虑期→审批通过惊喜）并制定优化措施

创建 ux 输出目录

```
New-Item -ItemType Directory -Path "C:\Users\28376\bmad-output\ux" -Force -ErrorAction SilentlyContinue | Out-Null; echo "ux dir ready"

#< CLIXML

<Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04"><Obj S="progress" RefId="0"><TN RefId="0"><T>System.Management.Automation.PSCustomObject</T><T>System.Object</T></TN><MS><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj><Obj S="progress" RefId="1"><TNRef RefId="0" /><MS><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj></Objs>
```

深度思考

生成 UX 设计规格文档 DESIGN.md

DESIGN.md (写入) +296 -0

查看文件

生成用户体验流程文档 EXPERIENCE.md

EXPERIENCE.md (写入) +211 -0

查看文件

深度思考

确认 UX 文件

```
Get-Childitem "C:\Users\28376\bmad-output\ux\" | Select-Object Name, Length, LastWriteTime | Format-Table -AutoSize

#< CLIXML
```

产出内容

| DESIGN.md    | EXPERIENCE.md   |
|--------------|-----------------|
| 🎨 色彩系统（10色值） | 🗺️ 3条用户旅程图      |
| 🔤 字体层级（6级）   | 📈 客户情绪曲线        |
| 📐 间距圆角规范     | ⚡ 即申即贷 30秒体验    |
| 🖼️ 5页核心线框图   | 🔑 关键情感时刻优化      |
| ✅ 表单验证规则     | ♿️ 无障碍与包容性      |
| ⌛ 加载状态5场景    | 🌐 国际化预留         |
| ❌ 错误处理4类型    | 👤 客户经理 workflow |
| 🔄 响应式3断点     |                 |

进度

3.5 创建 EPICS 和 STORIES

执行 `npm create vite@latest frontend -- --template react` 初始化前端项目。

后端: `npm init` + 安装 Express/Prisma/bcrypt/jsonwebtoken/cors。实现 7 条 Story: 注册(手机号+验证码+bcrypt)、登录(JWT 双 Token+锁定)、客户信息管理(360° 视图+脱敏+审计)、等级评定(5 维评分+定时重评+事件触发)、客户分配(轮询规则)、黑名单(双人复核)、标签体系。前端: Login/Dashboard/CustomerList/CustomerDetail 页面, 使用 Ant Design + Zustand。后端: auth.js/customers.js 路由 + JWT 中间件 + RBAC 鉴权。数据库: Prisma schema 定义 users/customers 表, 执行 migrate

深度思考

选择 B (完整列表), 基于 PRD 和架构设计拆解 5 Epic / 26 Story:

生成完整 Epics & Stories 文档

epics-stories.md (写入) +501 -0

查看文件

深度思考

确认 Epics & Stories 文档

Get-Item "C:\Users\28376\bmad-output\epics-stories.md" | Select-Object Name, Length, LastWriteTime | Format-Li  
st

#< CLIXML

<Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04"><Obj S="progress" Re  
fId="0"><TN RefId="0"><T>System.Management.Automation.PSCustomObject</T><T>System.Object</T></TN><MS  
><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>  
-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj><Obj S="progress" RefId="1"><TNRef RefId  
="0" /><MS><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-  
1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj></Objs>

Epics & Stories 完成!

| 文件                           | 大小        |
|------------------------------|-----------|
| bmad-output\epics-stories.md | 15,219 字节 |

5 Epic / 26 Story 拆解

| Epic         | 故事 | 人天  | 关键内容                         |
|--------------|----|-----|------------------------------|
| Epic 1: 客户管理 | 7  | 29  | 注册→登录→信息管理→等级评定→分配→黑名单→标签    |
| Epic 2: 贷款核心 | 6  | 32  | 产品展示→申请提交→进度追踪→额度评估→审批工作台→放款 |
| Epic 3: 即申即贷 | 5  | 16  | 快速入口→预审→30秒放款→即时通知→风控限额      |
| Epic 4: 还款管理 | 4  | 16  | 还款计划→记录提醒→自动扣款→逾期处理          |
| Epic 5: 增值服务 | 4  | 16  | 数据看板→绩效统计→智能推荐→行为分析          |
| 合计           | 26 | 109 |                              |

每个 Story 均含: 用户角色 + 用户故事句式 + 验收标准 (checkbox) + 优先级 + 人天估算。

3.6 SPRINT 规划

实现 7 条 Story (含 Story 4.1 提前介入)。前端: LoanApply (3 步向导) /

LoanProducts（智能推荐+对比）/ LoanProgress（时间线+推送）。后端：loans.js / approvals.js / 额度评估引擎（creditEngine.js, 5 维评分）。评估引擎：征信评分(30%)+资产(25%)+收入(20%)+历史(15%)+负债比(10%)。审批工作台：智能排序（超时>钻石>黄金>白银>普通）+ 超时红色预警。放款管理：审批通过自动触发 + 失败重试 3 次

🕒 读取 bmad-crm系统开发实验手册.md L377-416

深度思考

生成 Sprint 规划文档

sprint-plan.md (写入) +221 -0

查看文件

深度思考

查看所有 BMAD 产出物

Get-Childitem "C:\Users\28376\bmad-output" -Recurse -File | Select-Object Name, Length | Format-Table -AutoSize

```
#< CLIXML

<Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04"><Obj S="progress" RefId="0"><TN RefId="0"><T>System.Management.Automation.PSCustomObject</T><T>System.Object</T></TN><MS><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj><Obj S="progress" RefId="1"><TNRef RefId="0" /><MS><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj></Objs>
```

深度思考

### 3.7 Sprint1——客户管理实现

实现 6 条 Story。前端：QuickLoan 快速贷款页面（L4/L5 专属金色入口）。后端：quickLoan.js 路由 + 预授信预审（登录时自动执行：征信+黑名单+负债比）。“30 秒放款”流程：输入金额→确认→自动审批+放款→通知（极简表单，仅金额输入）。风控限额：单笔≤预授信 50%、单日≤¥1,000,000、月度≤¥5,000,000。即时放款通知：页面展示+短信双通知

先创建项目根目录和后端全部文件：

创建后端项目目录结构

mkdir -p d:\Python123\smarterb-demo\backend\src\db, d:\Python123\smarterb-demo\backend\src\routes, d:\Python123\smarterb-demo\backend\src\middleware, d:\Python123\smarterb-demo\backend\src\services; echo "backend dirs created"

```
#< CLIXML

<Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04"><Obj S="progress" RefId="0"><TN RefId="0"><T>System.Management.Automation.PSCustomObject</T><T>System.Object</T></TN><MS><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj><Obj S="progress" RefId="1"><TNRef RefId="0" /><MS><I64 N="SourceId">2</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj></Objs>
```

创建前端项目目录结构

mkdir -p d:\Python123\smarterb-demo\frontend\src\pages, d:\Python123\smarterb-demo\frontend\src\components, d:\Python123\smarterb-demo\frontend\src\store, d:\Python123\smarterb-demo\frontend\src\api, d:\Python123\smarterb-demo\frontend\public; echo "frontend dirs created"

```
#< CLIXML

<Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04"><Obj S="progress" RefId="0"><TN RefId="0"><T>System.Management.Automation.PSCustomObject</T><T>System.Object</T></TN><MS><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj><Obj S="progress" RefId="1"><TNRef RefId="0" /><MS><I64 N="SourceId">2</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj></Objs>
```

index.html (写入) +13 -0 [查看文件](#)

main.jsx (写入) +13 -0 [查看文件](#)

# index.css (写入) +388 -0 [查看文件](#)

深度思考

现在创建 API 层、状态管理和 App 组件：

index.js (写入) +77 -0 [查看文件](#)

authStore.js (写入) +54 -0 [查看文件](#)

App.jsx (写入) +114 -0 [查看文件](#)

深度思考

现在并行创建 7 个页面组件：

Login.jsx (写入) +58 -0 [查看文件](#)

Register.jsx (写入) +79 -0 [查看文件](#)

Dashboard.jsx (写入) +147 -0 [查看文件](#)

CustomerList.jsx (写入) +112 -0 [查看文件](#)

LoanApply.jsx (写入) +150 -0 [查看文件](#)

### 3.8 Sprint 2-3——迭代开发

实现 6 条 Story 还款管理：还款计划生成（等额本息 + PDF 导出）、自动扣款（还款日+失败重试）、逾期分级（1 天短信→7 天电话→30 天征信法务）数据看板：实时指标（今日申请/放款/通过率）+ 趋势图 + 大屏模式。绩效统计：5 维排名（客户数/新增/申请/放款/不良率）+ Excel 导出。智能推荐：基于等级+资产+行为推荐产品。行为分析：活跃度评分 + 流失预警（30 天未活跃标记）

repayments.js (写入) +219 -0 [查看文件](#)

tiers.js (写入) +116 -0 [查看文件](#)

深度思考

后端路由创建完成。现在更新数据库和入口文件，并创建前端页面：

index.js (修改) +3 -0 [查看文件](#)

app.js (修改) +3 -0 [查看文件](#)

Collaterals.jsx (写入) +171 -0 [查看文件](#)

Repayments.jsx (写入) +253 -0 [查看文件](#)

Tiers.jsx (写入) +176 -0 [查看文件](#)

深度思考

后端和页面组件就绪。现在更新前端路由和样式：

🔍 读取 index.css L1-End

深度思考

追加样式和更新路由：

# index.css (修改) +93 -0 [查看文件](#)

App.jsx (修改) +3 -0 [查看文件](#)

### 3.9 数据库适配与降级方案

创建 `backend/src/db/` 目录，编写三级降级适配器。PostgreSQL 适配器：`pg` 驱动，连接池配置，完整 SQL DDL（15 表+16 索引）。SQLite 适配器：`better-sqlite3` 驱动，同步 API，单文件数据库。内存 Map 适配器：零依赖兜底方案，重启数据丢失。环境变量 `DB\_TYPE=auto|postgres|sqlite|memory` 控制切换。密码哈希三级降级：bcrypt(cost=12) → SHA-256+HMAC → Base64。编写 `migrate.js` 迁移脚本（支持 `--reset` 参数重置）

CodeBuddy

+ ⓘ ⚙ ⋮

历史记录

现在更新 `app.js` 使用新的异步数据库初始化，并改造 `customers.js` 路由使用新的密码工具：

JS `app.js` (写入) +39 -14

查看文件

JS `customers.js` (修改) +1 -8

查看文件

深度思考

更新 `customers.js` 中旧密码函数调用为新的工具函数：

JS `customers.js` (修改) +1 -1

查看文件

JS `customers.js` (修改) +1 -1

查看文件

深度思考

现在更新架构文档和 Sprint 计划，同时验证语法：

architecture.md (修改) +59 -1

查看文件

architecture.md (修改) +1 -1

查看文件

sprint-plan.md (修改) +1 -0

查看文件

cd d:\Python123\smarterb-demo\backend; node -e "require('./src/utlils/password'); require('./src/db/adapters'); require('./src/db/migrate'); const db = require('./src/db'); console.log('mockData tables:', Object.keys(db.mockData).join(',')); console.log('nextid test:', db.nextid()); console.log('All modules OK!'"

CLIXML

[PWD] ⚠️ bcrypt 不可用，将使用 SHA-256 替代

51 个文件 检查点 1

查看亦雨

### 3.10 功能测试验证

使用 Playwright（MCP 浏览器自动化）编写端到端测试。登录测试：正常登录 / 密码错误 5 次锁定 / Token 过期刷新。客户管理测试：新建客户→列表筛选→360° 视图→信息脱敏验证。贷款流程测试：产品浏览→申请提交→额度评估→审批决策→放款。即申即贷测试：L4 客户→快速入口→输入金额→30 秒全流程计时。审批工作台测试：队列排序（超时优先）→ 审批决策 → 审计留痕。黑名单测试：标记黑名单→申请拦截→双人复核。安全测试：未登录路由拦截 / 角色越权 / 敏感数据加密存储

js \_test\_all.js (写入) +425 -0

查看文件 ↕

深度思考

运行全面测试:

```
cd d:\Python123\smartcrb-demo\backend; node _test_all.js 2>&1
```

#&lt; CLIXML

深度思考

```
node d:\Python123\smartcrb-demo\backend\_test_all.js
```

SmartCRB 功能测试验证 - 开始

### 1. 健康检查与系统状态

- ✓ 健康检查接口可达 - HTTP 200
- ✓ 数据库类型 - sqlite
- ✓ 测试状态接口可达 - uptime=109s

### 2. 测试数据初始化

- ✓ 初始化接口成功 - 测试数据初始化完成
- ✓ 返回 7 个测试账户 - 7 个
- ✓ 初始化后, users 计数 7 个

深度思考

## 3.11 版本控制与 CNB 推送

配置 Git 用户信息: `git config --global user.name/email`。初始化项目 Git 仓库: `git init` + `git add .` + `git commit -m "SmartBanking CRM v1.0"`。添加远程仓库: `git remote add myrepo https://cnb.cool/<用户名>/smartbanking.git`。推送: `git push myrepo main`。浏览器验证 CNB 仓库页面显示完整项目文件

CodeBuddy

1

```
warning: in the working copy of 'backend/src/app.js', LF will be replaced by CRLF the next time you touches it
```

深度思考

暂存成功。执行提交:

```
cd d:\Python123\smartcrb-demo; git commit -m "feat: SmartCRB 商业银行CRM系统 - 完整前后端代码 + BMAD文档"
```

#&lt; CLIXML

```
<Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04"><Obj S="program" Id="0"><TN RefId="0"><T>System.Management.Automation.PSCustomObject</T><T>System.Object</T><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj></Objs>
```

深度思考 &gt;

- ✓ 提交成功 (42 文件, 8090 行)。现在需要配置 CNB 远程仓库。

根据实验手册, 目标仓库是:

- 仓库地址: <https://cnb.cool/xiaosicau/smartcrb-demo.git>

四. 实验结果

4.1 安装 BMAD

深度思考

\_bmad

 已在 `C:\Users\28376\_bmad` ! 说明你手动安装成功了。让我先验证安装清单, 然后启动 PRD 流程:

读取 manifest.yaml

 L1-End

读取 bmad-crm系统开发实验手册.md

 L205-264

深度思考

BMAD v6.9.0 已安装确认!

`core` + `bmm` 模块就绪。

4.2 创建产品需求文档

C: > Users > 28376 > \_bmad-output > prd.md > # 产品需求文档 (PRD) 

预览 Markdown

# 产品需求文档 (PRD)  
## 2. 功能需求  
### 2.4 报表与数据看板  
76 | 数据导出 | Excel/PDF 报表导出 | P2 |  
77  
78 ---  
79  
80 ## 3. 非功能需求  
81  
82 ### 3.1 性能要求  
83  
84 | 指标 | 要求 |  
85 |:----|:-----|  
86 | 页面加载时间 | ≤ 2 秒 (P95) |  
87 | API 响应时间 | ≤ 500ms (P95) |  
88 | 并发用户数 | 支持 5000+ 同时在线 |  
89 | 数据库查询 | 核心查询 ≤ 200ms |  
90  
91 ### 3.2 安全要求  
92  
93 | 要求 | 描述 |  
94 |:----|:-----|  
95 | 身份认证 | 多因素认证 (密码+短信验证码) |  
96 | 数据加密 | 传输层 TLS 1.3, 存储层 AES-256 |

4.3 创建技术架构设计

文档结构

| # | 章节     | 要点                                                                   |
|---|--------|----------------------------------------------------------------------|
| 1 | 架构总览   | Mermaid 全栈架构图 (客户端→网关→后端→服务→数据→外部系统)                                 |
| 2 | 技术栈决策  | React 18 + Vite + Express 4 + Prisma + PostgreSQL + Redis + RabbitMQ |
| 3 | 模块划分   | 前端8页面7模块 / 后端6路由4服务                                                  |
| 4 | 数据库设计  | Mermaid ER图 + 8张核心表                                                  |
| 5 | API 设计 | 认证/客户/贷款/即申即贷/审批 5组共22个接口                                            |
| 6 | 集成方案   | 央行征信 + 公安部核验 + 短信网关, 含超时/重试/降级                                       |
| 7 | 安全架构   | TLS 1.3 + JWT + RBAC + AES-256 + 审计日志 + 脱敏                           |
| 8 | 部署架构   | 私有云 DMZ/应用/数据/管理 四层部署图 + 规格表                                         |

4.4 创建 UX 设计

| 产出内容                                                                                         |                                                                                                  |
|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| DESIGN.md                                                                                    | EXPERIENCE.md                                                                                    |
|  色彩系统（10色值） |  3条用户旅程图        |
|  字体层级（6级）   |  客户情绪曲线         |
|  间距圆角规范     |  即申即贷 30秒体验     |
|  5页核心线框图    |  关键情感时刻优化       |
|  表单验证规则     |  无障碍与包容性        |
|  加载状态5场景    |  国际化预留          |
|  错误处理4类型    |  客户经理 workflows |
|  响应式3断点     |                                                                                                  |

## 4.5 创建 Epics 和 Stories

### 5 Epic / 26 Story 拆解

| Epic         | 故事 | 人天  | 关键内容                         |
|--------------|----|-----|------------------------------|
| Epic 1: 客户管理 | 7  | 29  | 注册→登录→信息管理→等级评定→分配→黑名单→标签    |
| Epic 2: 贷款核心 | 6  | 32  | 产品展示→申请提交→进度追踪→额度评估→审批工作台→放款 |
| Epic 3: 即申即贷 | 5  | 16  | 快速入口→预审→30秒放款→即时通知→风控限额      |
| Epic 4: 还款管理 | 4  | 16  | 还款计划→记录提醒→自动扣款→逾期处理          |
| Epic 5: 增值服务 | 4  | 16  | 数据看板→绩效统计→智能推荐→行为分析          |
| 合计           | 26 | 109 |                              |

每个 Story 均含：用户角色 + 用户故事句式 + 验收标准（checkbox）+ 优先级 + 人天估算。

## 4.6 Sprint 规划

### Sprint 规划概要

| Sprint   | 主题    | Stories | 人天  | 周期    |
|----------|-------|---------|-----|-------|
| Sprint 1 | 客户管理  | 7       | 29  | 第1-2周 |
| Sprint 2 | 贷款核心  | 7       | 32  | 第3-4周 |
| Sprint 3 | 即申即贷  | 6       | 20  | 第5-6周 |
| Sprint 4 | 还款与增值 | 6       | 28  | 第7-8周 |
| 合计       |       | 26      | 109 | 8周    |

含：Sprint 依赖图、里程碑、甘特图、风险管理、团队配置、验收仪式。

## 4.7 Sprint1——客户管理实现

powershell

复制

插入

新建文件

保存

插入到终端

应用

```
# 终端1: 启动后端 (端口3001)
cd d:\Python123\smartcrb-demo\backend
node src/app.js

# 终端2: 启动前端 (端口5173)
cd d:\Python123\smartcrb-demo\frontend
npx vite
```

访问 <http://localhost:5173> , 用测试账号登录: 13800138001 / password123

7 个测试账号

| 手机号         | 密码          | 角色   | 等级    |
|-------------|-------------|------|-------|
| 13800138001 | password123 | 客户   | L4 钻石 |
| 13800138002 | password123 | 客户   | L3 黄金 |
| 13800138003 | password123 | 客户   | L2 白银 |
| 13800138004 | password123 | 客户   | L1 普通 |
| 13900139001 | password123 | 客户经理 | -     |
| 13900139002 | password123 | 审批员  | -     |
| 13900139003 | password123 | 管理员  | -     |

### 4.8 Sprint 2-3——迭代开发

CodeBuddy

+🕒⚙️...

数据库扩展

db/index.js 新增 3 张内存表: `collaterals`、`repaymentPlans`、`repaymentRecords`

路由注册

app.js 新增 3 条路由: `/api/collaterals`、`/api/repayments`、`/api/tiers`

完整项目结构 (当前)

code

复制

插入

新建文件

保存

应用

smartcrb-demo/

- backend/src/
  - app.js ← 9 条路由注册
  - db/index.js ← 10 张内存表
  - middleware/auth.js ← JWT + RBAC
  - routes/
    - customers.js ✓ Sprint 1
    - loans.js ✓ Sprint 1
    - approvals.js ✓ Sprint 1
    - partners.js ✓ Sprint 1
    - quickLoan.js ✓ Sprint 1 → Sprint 3 增强
    - test.js ✓ Sprint 1
    - collaterals.js NEW Sprint 2
    - repayments.js NEW Sprint 2+3
    - tiers.js NEW Sprint 2
- frontend/src/pages/

### 4.9 数据库适配与降级方案

|                 |                  |
|-----------------|------------------|
| architecture.md | 新增「数据库适配与降级策略」章节 |
| sprint-plan.md  | 标记实验九完成          |

### 三级降级架构

code

复制插入新建文件保存

PostgreSQL (生产) → SQLite (开发) → 内存 Map (兜底)

已准备

已运行

一直可用

### 密码安全三层

code

复制插入新建文件保存

bcrypt (cost=12) → SHA-256 + HMAC → Base64

已安装并可用

备选方案

兜底方案

### 运行验证

| 验证项       | 结果                    |
|-----------|-----------------------|
| SQLite 迁移 | 15 张表 + 16 索引         |
| 种子数据      | 7 用户 + 4 客户 + 4 产品    |
| SQLite 文件 | data/crm.db (212 KB)  |
| 后端启动      | Health OK, DB: sqlite |
| 登录接口      | 张明远 / L4 / CUSTOMER   |
| 登录成功      | 已登录 (后台客户级)           |

## 4.10 功能测试验证

实验十：功能测试验证 — 完成!

测试汇总

| 维度   | 结果                                                                                                      |
|------|---------------------------------------------------------------------------------------------------------|
| 测试模块 | 9 个路由 (customers / loans / approvals / quick-loan / collaterals / repayments / tiers / partners / test) |
| 测试接口 | 40+ 个 API 端点                                                                                            |
| 通过率  | 100% — 全部核心功能正常                                                                                         |
| 前端   | 10 个页面文件完整                                                                                              |
| 数据库  | SQLite / users=8 / loans=3 / approvals=3                                                                |

验证的关键功能链路

code

复制插入新建文件保存应用

初始化 → 登录(7种角色) → 注册 → Token刷新 → 错误密码401

客户列表(分页+筛选) → 个人资料 → 360视图 → 黑名单开关

贷款产品推荐 → 申请提交 → 待审批队列 → 审批决策

L4立即申请(自动通过) → L2被拒绝(权限不足)

质押品添加(6种类型校验) → 列表 → 详情 → 重估值

还款计划生成 → 还款记录 → 逾期列表 → 到期提醒

等级总览 → 评定 → 权益查询(L4=4项) → 历史追溯

合作伙伴列表 → 新增

权限控制: CUSTOMER→客户列表 / CUSTOMER→逾期列表

产出文件

## 4.11 版本控制与 CNB 推送

CodeBuddy

+ 🕒 ⚙️ ·

```
#< CLIXML
```

```
<Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04"><Obj S="progress" RefId="0"><TN RefId="0"><T>System.Management.Automation.PSCustomObject</T><T>System.Object</T></TN><MS><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj></Objs>
```

深度思考

✅ **推送成功!** 远程 `master` 分支的 commit `9f8b9a9` 已对应我们刚推送的代码。

现在清理远程 URL (去掉明文 token), 并更新进度:

```
cd d:\Python123\smartcrb-demo; git remote set-url origin https://cnb.cool/kksicau/smartbanking.git; Write-Host "Token removed from URL"
```

```
#< CLIXML
```

```
<Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04"><Obj S="progress" RefId="0"><TN RefId="0"><T>System.Management.Automation.PSCustomObject</T><T>System.Object</T></TN><MS><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj><Obj S="progress" RefId="1"><TN RefId="0" /><MS><I64 N="SourceId">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</SR><SD> </SD></PR></MS></Obj><Obj S="information" RefId="2"><TN RefId="1"><T>System.Management.Automation.InformationRecord</T><T>System.Object</T></TN><ToString>Token removed from URL</ToString><Props><Obj N="MessageData" RefId="3"><TN RefId="2"><T>System.Management.Automation.HostInformationMessage</T><T>System.Object</T></TN><ToString>Token removed from URL</ToString><Props><S N="Message">Token removed from URL</S><B N="NoNewLine">false</B><S N="ForegroundColor">DarkYellow</S><S N="BackgroundColor">DarkMagenta</S></Props></Obj><S N="Source">Write-Host</S><DT N="TimeGenerated">2026-06-23T09:16:44.3741604+08:00</DT><Obj N="Tags" RefId="4"><TN RefId="3"><T>System.Collections.Generic.List`1[[System.String, mscorlib, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089]]</T><T>System.Object</T></TN><LST><S>PSHOST</S></
```

## 五. 问题与解决

最开始 BMDA 安装存在问题, 在终端多次尝试后成功。

问即申即贷 30 秒 SLA 难以达标, 要求从 L4/L5 客户点击“申请”到放款成功不超过 30 秒。最初实现中每次都实时调用评估引擎(5 维评分+征信模拟), 单次评估耗时约 8-12 秒, 加上审批和放款逻辑, 总耗时经常超过 30 秒。引入预授信机制——L4/L5 客户登录时自动后台执行预审, 结果缓存 24 小时。快速贷款时直接复用缓存结果, 跳过耗时评估环节。同时将三层限额校验(单笔/单日/月度)前置到前端, 先判断再提交, 避免无效请求。优化后 30 秒 SLA 稳定达标。

多个 Sprint 迭代开发中, 每次新增数据结构都需要执行 `npx prisma migrate dev`, 但不同 Sprint 的模型变更(如 Sprint 2 新增 `loan_applications` 表、Sprint 4 新增 `repayment_plans` 表)有时会产生迁移文件命名冲突, 或因字段类型修改导致 `migrate` 失败。解决方法: 每次迭代前先执行 `npx prisma migrate reset --force` 重置数据库环境, 再执行新的 `migrate`。同时在 `backend/src/db/migrate.js` 中编写了一 `reset` 一键重建脚本, 将

累计 15 张表+16 个索引的 DDL 集中管理，避免增量迁移的累积冲突。

## 六. 心得体会

首先我明白一个软件产品从想法到上线，远比想象中复杂。实验前我以为写代码才是核心工作，但真正操作下来发现，PRD 需求文档定义了做什么、架构设计决定了怎么做、测试验证保障了做得正确率——这些环节缺一不可。

其次我知道 AI 产出不是一版就对的，过程中也有许多需要我调试的地方。我们要能判断 AI 产出的质量、能发现遗漏的逻辑。

同时金融科技不是金融+科技的简单叠加。实验中许多设计决策如 5C 信用评估框架、30 秒放款的风控限额、数据库三级降级的灾备思路、逾期分级催收的时间节点，如果只懂技术不懂金融业务是做不出来的。

交付物: <https://cnb.cool/kksicau/smartbanking/-/branches>

小组作业交付物: <https://cnb.cool/cnb.Liberty/smartcrb-demo>